



מכללת כנפי רוח - קרית נוער ירושלים

הצעת פרויקט לעבודת גמר

י"ד הנדסאי תוכנה (שאלון: 714918)

בנושא

סל חכם – מערכת סל קניות חכם דיגיטלית (SmartCart).



שם הסטודנט: דויד בן גיגי

ת"ז: 329235535

מנחה: אילן פרץ

תאריך: 13/1/2026

מכללה: כנפי רוח, קריית נוער ירושלים (סמל מוסד: 140129)

תוכן עניינים

1.	תיאור הנושא	3
2.	רקע תיאורטי	3
3.	תיאור הפרויקט	6
4.	הגדרת הבעיה האלגוריתמית	8
5.	הליכים עיקריים בפתרון בעיה בטכנולוגיות הנדסה מתקדמות	11
6.	הליכים עיקריים בתחום למידת מכונה – לא רלוונטי	14
7.	הליכים עיקריים במערכות הפעלה/ רשתות/תקשורת/אבטחה – לא רלוונטי	14
8.	תיאור פרוטוקולי תקשורת	15
9.	פיתוחים עתידיים	16
10.	תיאור טכנולוגיות הנדסה	17
11.	מסד נתונים	18
12.	פרטים פורמליים	20

1. תיאור הנושא

התחום שבו עוסקת עבודה זו הינו מסחר אלקטרוני (E-Commerce), המהווה כיום נדבך מרכזי בכלכלה העולמית ומתמקד בביצוע עסקאות מסחריות באמצעים דיגיטליים. המטרה העיקרית של תחום זה היא להנגיש מגוון רחב של מוצרים ושירותים לצרכן הסופי בכל זמן ומקום, לייעל את תהליך הרכישה ולאפשר השוואת מחירים ומוצרים בצורה נוחה ומהירה. אחד המאפיינים הבולטים של התחום בעידן המודרני הוא היקף הנתונים העצום והשפע האינסופי של הקטלוגים הדיגיטליים, המציעים לצרכן אלפי אפשרויות בחירה בלחיצת כפתור. עם זאת, מאפיין השפע מציב בפני התחום אתגרים ובעיות משמעותיות, ובראשם תופעה פסיכולוגית וצרכנית המכונה "הצפת מידע". האתגר המרכזי הקיים כיום בתחום הוא הקושי הקוגניטיבי והחשובי של המשתמש לקבל החלטות מושכלות מול היצע כה רחב, כאשר הצרכן הממוצע מתקשה לבצע שקלול אופטימלי של משתנים רבים (כגון מחיר, איכות וסגנון) מול אילוצי התקציב שלו, קושי שמוביל לעיתים קרובות לתסכול צרכני ולנטישת תהליך הקנייה.

2. רקע תיאורטי

הבעיה העומדת בבסיס התחום אינה עומדת בפני עצמה, אלא מהווה חלק מענף רחב ומרכזי במדעי המחשב ובמתמטיקה הנקרא אופטימיזציה קומבינטורית. על פי הגדרתה, אופטימיזציה קומבינטורית עוסקת במציאת אובייקט אופטימלי מתוך קבוצה סופית של אובייקטים (ויקיפדיה: אופטימיזציה קומבינטורית). בבעיות אלו, הנדרש הוא למצוא את הפתרון הטוב ביותר (מקסימום או מינימום) לפונקציית מטרה מסוימת, תחת אילוצים נתונים, כאשר מרחב הפתרונות הוא בדיד. דוגמאות מפורסמות לתחום זה כוללות את "בעיית הסוכן הנוסע" ו"בעיית הכיסוי", אך הבעיה הרלוונטית ביותר לתחום היעול הצרכני היא בעיית התרמיל (Knapsack Problem). בגרסתה הקלאסית והנפוצה ביותר, "תרמיל הגב הבינארי" (0/1), נתונה קבוצת n עצמים שלכל אחד מהם משקל ושווי. המטרה היא למצוא תת-קבוצה של פריטים הממקסמת את סך השווי, תחת האילוץ שסך המשקלים לא יעלה על חסם W (ויקיפדיה: בעיית תרמיל הגב). חשיבותה המעשית של בעיה זו באה לידי ביטוי במגוון רחב של מערכות קיימות: מערכות תשתית כגון Kubernetes משתמשות בווריאציות של הבעיה כדי לשבץ אפליקציות על שרתים פיזיים תחת מגבלות זיכרון וכוח עיבוד, ופלטפורמות למסחר פיננסי אוטומטי משתמשות באלגוריתם דומה לבניית סל השקעות הממקסם רווח תחת מגבלת סיכון ותקציב. עם זאת, הבעיה המאפיינת מערכות מסחר מודרניות מורכבת יותר מהגרסה הקלאסית, כיוון שהיא כוללת לרוב מספר אילוצים בו-זמנית (כגון מחיר וכמות פריטים). בעיה זו מוכרת

בספרות האקדמית כ- MDKP (Multidimensional Knapsack Problem). ההבדל המרכזי הוא שבמקום וקטור משקל יחיד, לכל פריט יש וקטור משאבים. בעיה זו מסווגת כ- NP-Hard על פי (SCIRP, 2018), ומשמעות הדבר התיאורטית היא שלא ידוע על אלגוריתם הפותר אותה בזמן פולינומיאלי לכל קלט אפשרי. כדי להתמודד עם מורכבות זו, נבחנו בספרות שלוש גישות אלגוריתמיות עיקריות. הטבלה הבאה משווה ביניהן ומציגה את הפערים הקיימים.

להלן טבלת השוואה המנתחת את הפתרונות הקיימים ומסבירה את הפער ביניהם.

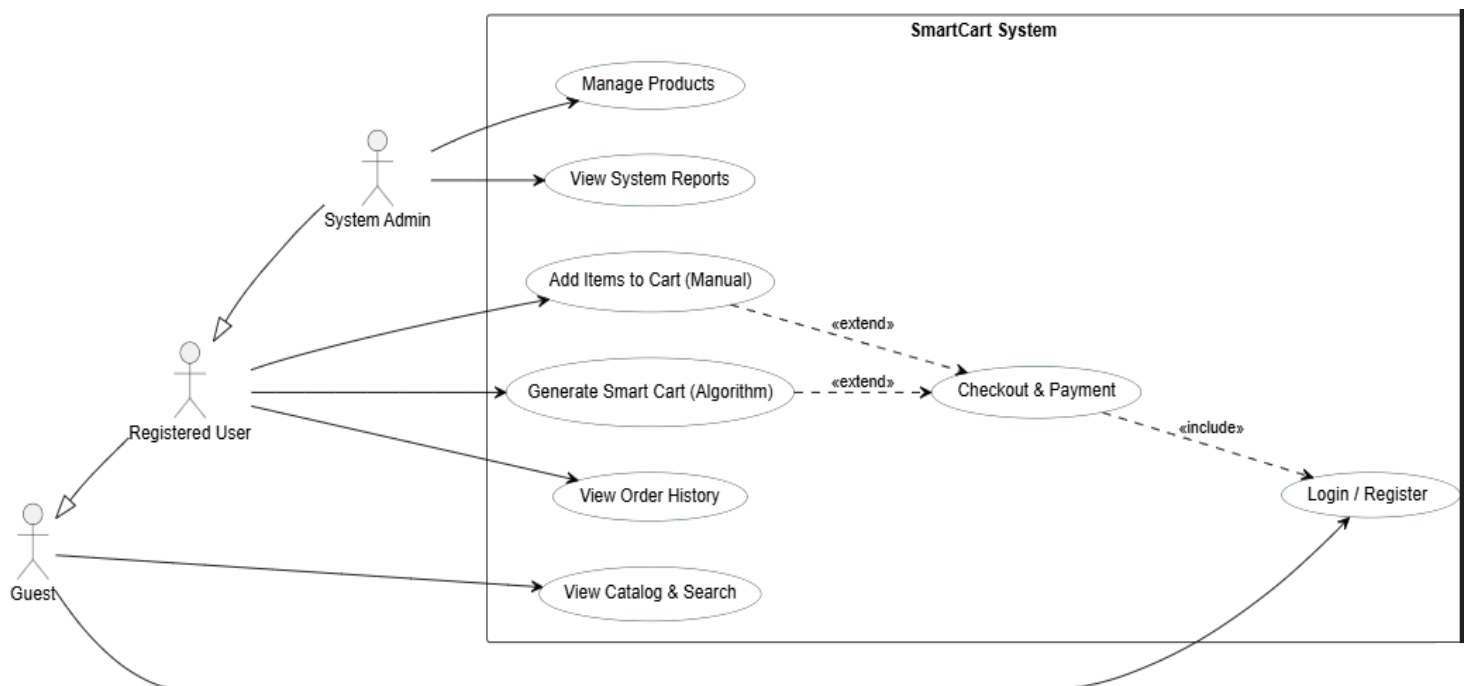
שם הגישה / אלגוריתם	מנגנון הפעולה	יתרונות	חסרונות	הפער בפתרונות הקיימים
אלגוריתם חמדן	בחירה בכל צעד בפריט עם היחס עלות/תועלת הטוב ביותר באותו רגע.	זמן ריצה מהיר מאוד $O(N \log N)$ וקל למימוש תכנותי.	אינו מבטיח פתרון אופטימלי ועלול להשאיר "חורים" לא מנוצלים בתקציב.	חוסר דיוק: בבעיות פיננסיות, פתרון שאינו אופטימלי (קירוב) נתפס כחיסרון משמעותי שאינו ממצה את משאבי הלקוח.
כוח גס (Brute Force)	סריקת כל הקומבינציות האפשריות של מוצרים בקטלוג כדי למצוא את הטובה ביותר.	מבטיח ב-100% את מציאת הפתרון האופטימלי (הטוב ביותר האפשרי).	זמן ריצה מעריכי שגדל בצורה קיצונית עם כל פריט נוסף.	חוסר יעילות: השיטה אינה ישימה במערכות זמן-אמת עם קטלוגים גדולים, שכן זמן החישוב יהיה ארוך מדי.

תכנות דינמי	פירוק הבעיה לתתי-בעיות קטנות ושמירת תוצאותיהן בטבלה למניעת חישובים חוזרים.	מבטיח פתרון אופטימלי מוחלט (כמו כוח גס) אך בזמן יעיל משמעותית (פסאודו-פולינומיאלי).	צריכת זיכרון התלויה בגודל הטבלה (מכפלת האילוצים).	הפתרון האידיאלי: זוהי הגישה היחידה המאפשרת דיוק מתמטי מושלם בזמן ריצה סביר עבור ערכים מספריים שלמים בלבד.
אלגוריתמי קירוב	משפחת אלגוריתמים המבטיחים מציאת פתרון הקרוב לאופטימלי עד כדי שגיאה ידועה מראש (למשל FPTAS).	זמן ריצה פולינומיאלי, מתאים מאוד לבעיות Big Data עם מיליוני פריטים.	הפתרון אינו אופטימלי ב-100% אלא רק "קרוב מספיק". המימוש לעיתים מורכב מאוד	כאשר הקלטים הם בטווח סביר (כמו במערכות מסחר רגילות), אין הצדקה להתפשר על דיוק לטובת מהירות, שכן ניתן להשיג את שניהם.

לאור הסקירה המוצגת, הספרות האקדמית מצביעה על גישת **התכנות הדינמי** כפתרון המתאים ביותר לבעיות MDKP בעלות אילוצים מספריים בדידים. גישה זו מאפשרת לאזן בצורה האופטימלית בין הדרישה לדיוק מתמטי מוחלט לבין מגבלות זמן הריצה, ולכן היא נחשבת לסטנדרט המקצועי המועדף לפתרון אתגרים אלגוריתמיים מסוג זה במערכות מחשוב מתקדמות.

3. תיאור הפרויקט

פרויקט "סל חכם (SmartCart)" הינו מערכת Web מלאה (Full-Stack) בתחום האופנה, אשר פותחה במטרה לספק מענה טכנולוגי לבעיית הצפת המידע והקושי בקבלת החלטות שתוארו בפרק הקודם. המערכת משמשת כחנות וירטואלית מקיפה, המאגדת בתוכה את כלל הכלים הסטנדרטיים המצופים מאתר מסחר מודרני, לרבות מודול לניהול משתמשים המאפשר הרשמה והתחברות לשמירת פרופיל אישי, קטלוג מוצרים דינמי ועשיר הכולל תמונות ומפרטים, מנועי חיפוש וסינון מתקדמים, וניהול סל קניות ידני לביצוע רכישות שגרטיות. מעבר לפונקציונליות הבסיסית, חדשנותו של הפרויקט באה לידי ביטוי בשילוב רכיב "הסל החכם", המשמש כעוזר קנייה אישי המבוסס על אלגוריתמיקה מתקדמת. רכיב זה נועד להחליף את תהליך החיפוש הידני והמייגע, ומאפשר למשתמש להגדיר בצורה פשוטה סט של אילוצים והעדפות אישיות, כגון תקציב מקסימלי, טווח כמות פריטים רצויה וסגנון לבוש מועדף. ברגע שהמשתמש מזין את נתוניו, המערכת מבצעת עיבוד נתונים מהיר ובונה עבורו באופן אוטומטי את סל הקניות האופטימלי ביותר העומד בכל הדרישות שהוגדרו, ובכך מגשרת על הפער בין ההיצע העצום לבין יכולת העיבוד המוגבלת של הצרכן. שילוב זה בין חנות אונליין מסורתית לבין מנוע אופטימיזציה חכם מעניק למשתמש חווית קנייה יעילה, חוסך זמן חיפוש יקר ומבטיח ניצול כלכלי מיטבי של התקציב העומד לרשותו. להלן תרשים הממחיש את האינטראקציה בין סוגי המשתמשים השונים לבין רכיבי המערכת, תוך הצגה ויזואלית של ההיררכיה והפעולות האפשריות.



התרשים מציג היררכיה של שלושה סוגי משתמשים, כאשר כל אחד יורש את היכולות של קודמו:

- **אורח: (Guest)** משתמש אנונימי שנכנס לאתר. הוא יכול רק להתרשם מהמוצרים ולחפש פריטים, אך אינו יכול לבצע רכישה ללא הזדהות.
- **משתמש רשום: (Registered User)** הלקוח המרכזי במערכת. יש לו את כל יכולות האורח, ובנוסף הוא יכול לנהל סל קניות (ידני או חכם), לצפות בהיסטוריית ההזמנות שלו ולבצע תשלום.
- **מנהל מערכת: (System Admin)** משתמש בעל הרשאות-על. תפקידו אינו לקנות, אלא לתחזק את המערכת: להוסיף מוצרים חדשים לקטלוג, לעדכן מחירים ולצפות בדוחות על פעילות האתר.

הסבר קצר על התהליכים המרכזיים במערכת:

1. **View Catalog & Search (צפייה וחיפוש):** פעולה בסיסית המאפשרת למשתמש לדפדף בין המוצרים, לראות תמונות ומחירים ולהשתמש בפילטרים (כגון צבע, שם או מידה) כדי לאתר פריט ספציפי.
2. **Login / Register (הרשמה והתחברות):** תהליך יצירת חשבון או כניסה לחשבון קיים. פעולה זו הכרחית כדי שהמערכת תדע למי לשייך את ההזמנה ואת הנתונים האישיים.
3. **Add Items to Cart - Manual (הוספה ידנית):** תהליך הקנייה הקלאסי. המשתמש בוחר פריטים בודדים שהוא אוהב ומוסיף אותם לסל הקניות שלו.
4. **Generate Smart Cart (יצירת סל חכם):** הפעולה הייחודית של המערכת. המשתמש מפעיל את האלגוריתם, מזין את הדרישות שלו, ומקבל תוצאה מחושבת של סל שלם.
5. **Checkout & Payment (ביצוע הזמנה):** שלב הסיום. המשתמש מאשר את הסל (בין אם נבנה ידנית ובין אם על ידי האלגוריתם), מזין פרטי תשלום ומשלוח, וההזמנה נשמרת במסד הנתונים. פעולה זו מחייבת שהמשתמש יהיה מחובר.
6. **Manage Products (ניהול מוצרים):** ממשק המיועד למנהל בלבד, המאפשר הוספה, עריכה או מחיקה של פריטים מהחנות (למשל, עדכון מלאי או שינוי מחיר).

תיאור תרחיש - יצירת סל חכם: מכיוון שהפעולה של "סל חכם" היא המורכבת והחדשנית ביותר, נתאר כיצד היא נראית מצד המשתמש: המשתמש נכנס למסך הייעודי ומגדיר את האילוצים שלו: "אני רוצה להוציא עד 600 ש"ח, ולקנות בדיוק 4 פריטים בסגנון ספורטיבי". לאחר לחיצה על כפתור "צור סל", המערכת מעבדת את הבקשה (בזמן שהמשתמש רואה חיווי טעינה), סורקת את הקטלוג ומוצאת את הקומבינציה המשתלמת ביותר. בסיום, מוצג למשתמש סל מוכן עם 4 הפריטים שנבחרו, והוא יכול להעביר את כולם לקופה בלחיצת כפתור אחת, ללא צורך לחפש אותם בנפרד.

4. הגדרת הבעיה האלגוריתמית

כפי שתואר בפרק הראשון (תיאור הנושא), האתגר המרכזי בפרויקט הוא לגשר על הפער בין הרצון של המשתמש למקסם את התמורה שהוא מקבל מבחינת איכות והתאמה אישית, לבין המגבלות הפיננסיות והכמותיות שלו. בעוד שבצד הפסיכולוגי הבעיה מוגדרת כ"הצפת מידע", בצד הטכנולוגי היא מתורגמת לבעיה חישובית של סריקת מרחב פתרונות עצום. באופן ידני, בדיקת כל הצירופים האפשריים של פריטים מתוך קטלוג גדול היא משימה בלתי אפשרית. לכן, המערכת נדרשת לבצע תהליך אוטומטי של קבלת החלטות. עליה לסרוק את הנתונים, לשקלל את הערך של כל פריט עבור המשתמש הספציפי, ולבחור את ההרכב המנצח.

מבחינה מדעית, הליבה של הפרויקט מוגדרת כווריאציה של בעיית תרמיל הגב הרב ממדי (MDKP). נתון לנו קטלוג המכיל N פריטים (בגדים), לאחר שעברו סינון ראשוני. כל פריט i בקטלוג זה מאופיין על ידי שני ערכים מספריים קבועים. האחד הוא המחיר (P) , המייצג את העלות לצרכן, והשני הוא הניקוד (S) , המייצג את "ציון ההתאמה" והתועלת למשתמש.

המערכת מקבלת מהמשתמש קלט הכולל שלושה אילוצים קשיחים. תקציב מקסימלי (W) , כמות פריטים מינימלית $(K_{\min})$ וכמות פריטים מקסימלית $(K_{\max})$.

כדי לפתור את הבעיה, אנו מגדירים עבור כל פריט "משתנה החלטה" בינארי (X) , אשר יכול לקבל את הערך 1 (הפריט נבחר) או 0 (הפריט לא נבחר).

המטרה האלגוריתמית היא למצוא את וקטור ההחלטות שימקסם את סכום הניקוד הכולל, כפי שמוצג בנוסחה הבאה:

$$\text{Maximize } \sum_{i=1}^N S_i \cdot x_i$$

תהליך מקסום זה אינו חופשי, אלא כפוף לשלושה אילוצים מתמטיים מחייבים שהוגדרו במערכת. הראשון הוא **אילוץ התקציב**, הדורש שסכום המחירים לא יחרוג מהתקרה שהוגדרה.

$$\sum_{i=1}^N P_i \cdot x_i \leq W$$

השני הוא **אילוץ הכמות**, הדורש שמספר הפריטים שנבחרו יהיה בטווח המדויק.

$$K_{min} \leq \sum_{i=1}^N x_i \leq K_{max}$$

והשלישי הוא **אילוץ הבחירה**, המגדיר שהבעיה היא בדידה (0/1) ושכל פריט הוא יחידה אחת שלמה.

$$x_i \in \{0, 1\}$$

מתוך הגישות שנבחנו בסקירה התיאורטית, הפתרון ההנדסי שנבחר למימוש בפרויקט הוא אלגוריתם התכנות הדינמי. הבחירה בגישה זו, על פני אלגוריתמים חמדניים או יוריסטיים, נובעת מהדרישה החד-משמעית לדיוק. במערכת מסחר המנהלת כסף של לקוחות, סטייה מהפתרון האופטימלי (גם אם קטנה) עלולה לפגוע באמינות המערכת.

האתגר המרכזי בשימוש בתכנות דינמי הוא סיבוכיות הזמן הפסאודו-פולינומיאלית שלו. בפתרון המורחב שפיתחנו (הכולל ממד שלישי לכמות פריטים), סיבוכיות הזמן היא $O(N*W*K)$. באופן תיאורטי, עבור קלטים בעלי ערכים מספריים עצומים (למשל תקציב W של מיליארדים), האלגוריתם אינו יעיל. אולם, בניתוח ההנדסי של פרויקט "סל חכם", מצאנו שהשיטה ישימה ואף אידיאלית, מהסיבות הבאות:

במערכת חנות בגדים, מחירי הפריטים והתקציב הם מספרים חסומים וסבירים (מאות עד אלפי שקלים), וכמות הפריטים בסל (K) היא חד-ספרתית.

תחת טווח ערכים זה, האלגוריתם מתכנס לפתרון תוך שברירי שנייה, ומספק חווית משתמש חלקה ללא עיכובים מורגשים.

לפיכך, הפתרון שנבחר מאזן בצורה המיטבית בין הדיוק המתמטי לבין דרישות הביצועים של מערכת Web מודרנית.

תהליך פתרון הבעיה במערכת מתבצע באמצעות יישום הנדסי של אלגוריתם התכנות הדינמי המורחב. התהליך מחולק לחמישה שלבים עוקבים, המבטיחים דיוק מקסימלי ויעילות חישובית:

שלב 1 - השלב הראשון מתחיל מיד עם קבלת האילוצים מהמשתמש (תקציב W , טווח כמויות $K_{\min}$, $K_{\max}$ ותגיות סגנון). המערכת מבצעת סינון ראשוני של קטלוג המוצרים ושולפת רק את הפריטים הרלוונטיים לסגנון ולמלאי הזמין, במטרה להקטין את מרחב החיפוש (N). האלגוריתם בשרת עובד בשיטת "התאמה מצטברת" ככל שלפריט יש יותר תגיות שתואמות את ההעדפות המדויקות של המשתמש, כך הניקוד שלו עולה. פריט עם תיאור דל יקבל ניקוד נמוך (כי רמת הוודאות לגביו נמוכה), בעוד פריט עם תיאור עשיר התואם את פרופיל המשתמש, יקבל ניקוד גבוה שיקנה לו סיכוי סביר מאד להיכנס ל"סל החכם". לאחר מכן, מתבצעת פעולה קריטית של נרמול נתונים: מכיוון שאלגוריתם התכנות הדינמי מחייב שימוש במספרים שלמים בלבד עבור האינדקסים של הטבלה, המערכת ממירה את כל מחירי הפריטים ואת התקציב הכולל לייצוג של אגורות (מספרים שלמים), ובכך מונעת שגיאות חישוב.

שלב 2 - בשלב זה המערכת מקצה את משאבי הזיכרון הנדרשים לחישוב. נוצרת טבלה תלת-ממדית בגודל $[N+1][W+1][K_{\max}+1]$. כל תא בטבלה זו, המסומן כ- $dp[i][w][k]$, מייצג את הניקוד המקסימלי שניתן להשיג תחת הרכב ספציפי של פריטים, תקציב וכמות. כל התאים בטבלה מאותחלים לערך התחלתי (כגון 0 או מינוס אינסוף) המייצג מצב שטרם חושב, למעט תא הבסיס (0 פריטים ב-0 תקציב) שמקבל ניקוד 0.

שלב 3 - זהו לב האלגוריתם. המערכת עוברת בלולאה על כל פריט i בקטלוג המסווג, ועבורו סורקת את כל הקומבינציות האפשריות של תקציב (w) וכמות (k). עבור כל תא בטבלה מתקבלת החלטה אופטימלית בזמן אמת המבוססת על יחס הנסיגה, המערכת בודקת האם עדיף "לא לקחת" את הפריט (ולהעתיק את הערך מהשלב הקודם), או "לקחת" אותו (להוסיף את הניקוד שלו לפתרון האופטימלי שכבר חושב עבור יתרת התקציב והכמות). הערך הגבוה מבין השניים נשמר בתא.

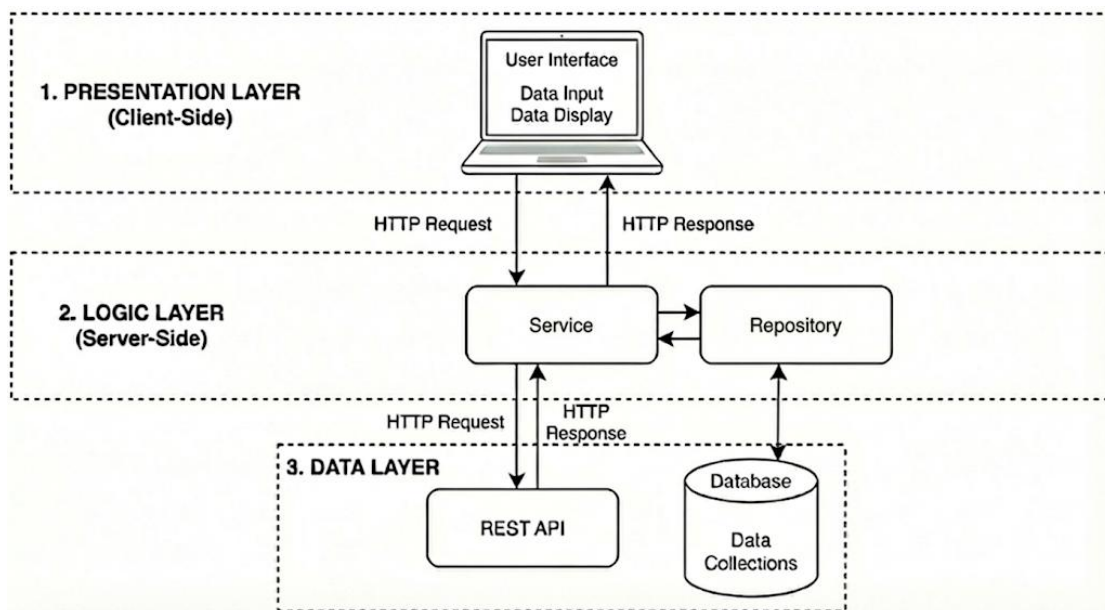
בפרויקט שלנו, לוגיקה זו מורחבת לממד שלישי (k) כדי להבטיח עמידה גם באילוץ כמות הפריטים. זמן הריצה של שלב זה הוא $O(N * W * K_{\max})$, המוגדר כפסאודו-פולינומיאלי, משמעות הדבר היא שפתרון זה יעיל מאוד עבור ערכים צרכניים סבירים (כמו במערכת שלנו), אך עבור קלטים שבהם התקציב (W) או הכמות (K) הם מספרים גדולים מאוד (אסטרונומיים), האלגוריתם הופך ללא-יעיל ולא יוכל לפתור את הבעיה בזמן סביר, שכן גודל הטבלה בזיכרון תלוי בערך המספרי של הקלט.

שלב 4 - בסיום מילוי הטבלה, המערכת מחזיקה את הניקוד המקסימלי לכל כמות אפשרית. כיוון שהמשתמש הגדיר טווח של פריטים (למשל בין 3 ל-5), המערכת אינה מסתפקת בבדיקת תא בודד. היא סורקת את כל התאים בשורה האחרונה של הטבלה (המייצגת את כלל הפריטים והתקציב המלא) אשר נמצאים בתוך טווח הכמויות המבוקש. מבין כל האפשרויות החוקיות הללו, נבחר התא בעל הניקוד הגבוה ביותר, המייצג את "הסל המושלם".

שלב 5 - לאחר שאותר הציון המקסימלי, המערכת נדרשת להבין אילו פריטים ספציפיים הרכיבו אותו. לשם כך מתבצע תהליך של "הליכה לאחור" מהתא הנבחר ועד לתחילת הטבלה. בכל צעד, המערכת בודקת האם הערך בתא הנוכחי שונה מהערך בתא המקביל לו בשלב הקודם (בלי הפריט). אם יש שוני, משמעות הדבר שהפריט נבחר לסל, והוא מתווסף לרשימת התוצאות הסופית תוך הפחתת מחירו וכמותו מהיתרה. בסיום התהליך, רשימת הפריטים המדויקת נשלחת בחזרה לממשק המשתמש להצגה.

5. הליכים עיקריים בפתרון בעיה בטכנולוגיות הנדסה מתקדמות

המערכת תוכננה ופותחה על בסיס ארכיטקטורת "מודל שלוש השכבות", מתוך מטרה לייצר הפרדת רשויות מלאה בין ממשק המשתמש, הלוגיקה העסקית ותשתית הנתונים.



להלן תיאור מעמיק של כל שכבה:

1. שכבת התצוגה

שכבה זו מהווה את החזית הטכנולוגית של המערכת ורצה כולה בסביבת הדפדפן של המשתמש. תפקידה הוא לשמש כנקודת המפגש האינטראקטיבית בין האדם למכונה, היא אחראית על הרינדור הוויזואלי, ניהול חווית המשתמש ואיסוף כלל הקלטים, החל מתהליכי הזדהות ועד לביצוע פעולות עסקיות במערכת. מבחינת תקשורת, השכבה מתפקדת כ"לקוח" היוזם את הפניות לשרת, בכל פעולה של המשתמש, השכבה אורזת את המידע לאובייקטים בפורמט JSON ושולחת בקשות רשת אסינכרוניות בפרוטוקול HTTP לעבר שכבת הלוגיקה, ועם קבלת התשובה היא מעדכנת את התצוגה בזמן אמת.

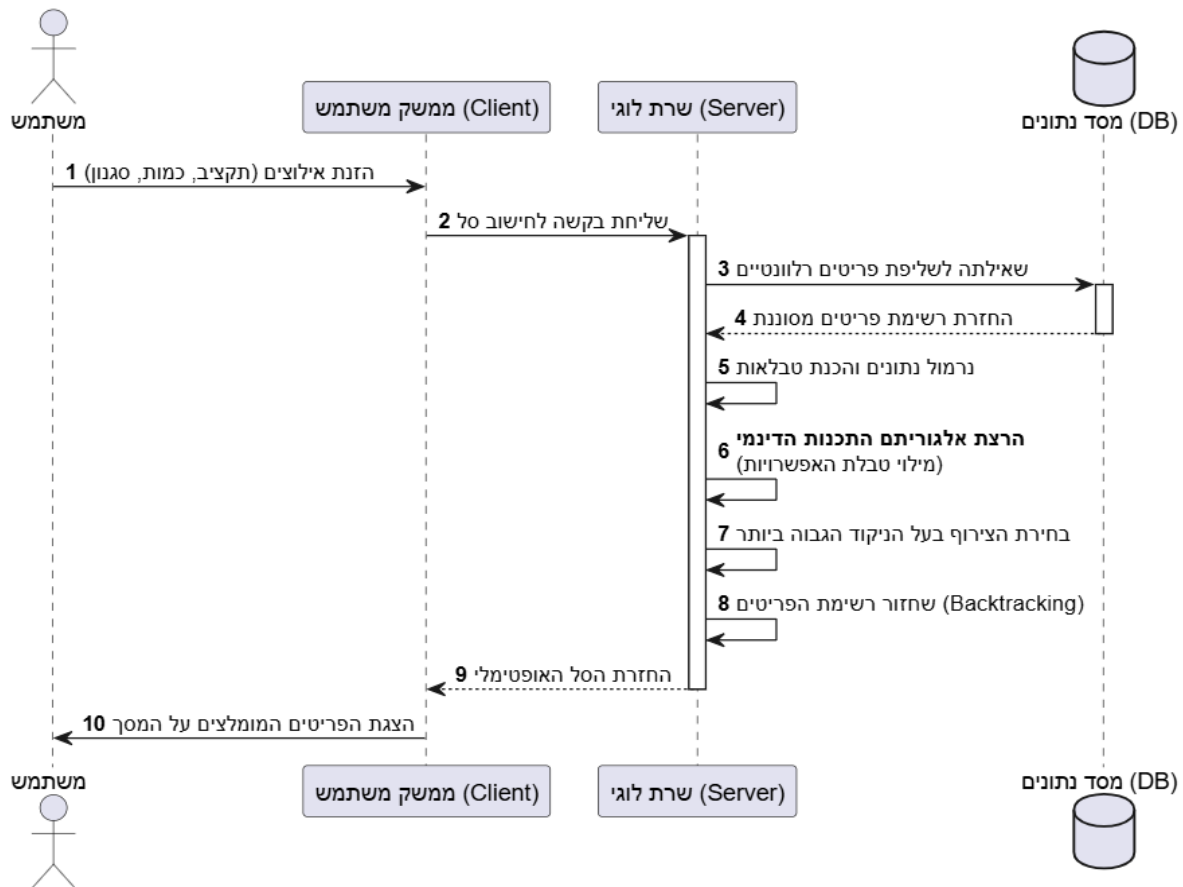
2. שכבת הלוגיקה והשירותים

ליבת המערכת מתבצעת בצד השרת, בסביבת ריצה ייעודית בשם Apache Tomcat כחלק מ Spring Boot ומרכזת את כלל החוקיות העסקית. בתוך שכבה זו פועלים שני רכיבי תוכנה מרכזיים, הלב הוא רכיב השירות (Service), אשר קולט את בקשות ה-HTTP, מפענח אותן ומריץ את האלגוריתמים המורכבים של המערכת. כדי לבצע את משימתו, רכיב השירות נעזר ברכיב המאגר (Repository). תפקידו של רכיב המאגר הוא לשמש כמתווך טכני לנתונים ולתרגם את הבקשות הלוגיות לשאילתות מסד נתונים. במקביל, רכיב השירות מסוגל ליזום תקשורת ישירה החוצה לשירותי צד-שלישי (REST API) לצורך העשרת המידע, ללא תלות במסד הנתונים הפנימי.

3. שכבת הנתונים והאינטגרציה

השכבה השלישית מייצגת את תשתית המידע החיצונית שאליה מתחבר השרת. בתרשים המערכת, שכבה זו כוללת שני ערוצים נפרדים, הערוץ המרכזי הוא מסד הנתונים, היושב על שרת אחסון נפרד ומנהל את שמירת המידע הקבוע באוספים שונים, אליו ניגשת המערכת אך ורק דרך רכיב ה-Repository. במקביל, קיים ערוץ אינטגרציה מול ממשקי API חיצוניים, המאפשר לרכיב השירות (Service) לצרוך נתונים בזמן אמת משירותים בענן. הגישה המשולבת לשני מקורות המידע הללו מאפשרת למערכת לספק למשתמש תמונת מצב מלאה ואמינה.

כדי להמחיש את האינטראקציה בין שכבת התצוגה, הלוגיקה והנתונים שתוארו לעיל, מוצג להלן תרשים המתאר את התהליך המורכב ביותר במערכת, חישוב הסל האופטימלי. התרשים ממפה את סדר הפעולות ואת חילופי המסרים בין השכבות השונות במערכת.



התרשים לעיל מציג את ההתנהגות הדינמית של המערכת בעת הרצת הפיצ'ר המרכזי "יצירת סל חכם". התהליך מתאר את שרשרת האירועים מרגע שהמשתמש מזין את בקשתו ועד קבלת התוצאה, תוך פירוט האינטראקציה בין שכבת התצוגה, השרת הלוגי ומסד הנתונים.

להלן פירוט שלבי התהליך:

1. שלב הייזום ושליחת הבקשה (צעדים 1-2): התהליך מתחיל בממשק המשתמש (Client), כאשר המשתמש מזין בטופס את אילוצי הסל הרצויים, תקציב מקסימלי, טווח כמות פריטים וסגנון מועדף. לאחר שהלקוח לוחץ על כפתור החישוב, שכבת התצוגה אוספת את הנתונים ושולחת בקשת HTTP לשרת, המכילה את כל הפרמטרים הללו לעיבוד.

2. שלב שליפת הנתונים וסינון ראשוני (צעדים 3-4): השרת הלוגי (Server) מקבל את הבקשה ופונה למסד הנתונים בשאלתה ממוקדת. מטרת השאלתה היא לשלוף רק את הפריטים הרלוונטיים (למשל, רק פריטים מהסגנון שנבחר ושיש להם מלאי זמין), וזאת על מנת לצמצם את מרחב החיפוש ולהקטין את העומס החישובי בהמשך. מסד הנתונים מחזיר לשרת רשימה מסוננת של פריטים.

3. שלב העיבוד האלגוריתמי (צעדים 5-8): זהו לב המערכת, המתבצע כולו בתוך השרת הלוגי. לאחר קבלת הפריטים, השרת מבצע את סדרת הפעולות הבאות:

- **נרמול נתונים:** המרת המחירים למספרים שלמים והכנת המגבלות לפורמט המתאים לאלגוריתם.
- **הרצת האלגוריתם:** בניית טבלת התכנות הדינמי ומילוי האפשרויות השונות (חישוב הניקוד המקסימלי לכל תת-תקציב).
- **בחירת הצירוף:** איתור התא בטבלה המייצג את הניקוד הגבוה ביותר תחת האילוצים.
- **Backtracking:** שחזור רשימת הפריטים הספציפיים שהרכיבו את הצירוף המנצח.

4. שלב התגובה והתצוגה (צעדים 9-10): לאחר שהסל הורכב בהצלחה, השרת הלוגי אורז את התוצאה (רשימת הפריטים והמחיר הסופי) ושולח אותה חזרה לממשק המשתמש כתשובת HTTP. הממשק קולט את המידע ומציג למשתמש את הפריטים המומלצים בצורה ויזואלית על המסך.

6. הליכים עיקריים בתחום למידת מכונה – לא רלוונטי

הצעה זו לא עוסקת בלמידת מכונה, לכן סעיף זה לא רלוונטי.

7. הליכים עיקריים במערכות הפעלה/רשתות/תקשורת/אבטחה – לא רלוונטי

הצעה זו לא עוסקת במערכות הפעלה / רשתות / תקשורת / אבטחת מידע.

לכן סעיף זה לא רלוונטי.

הערה חשובה: ההתייחסות למודל Client/Server מופיעה בהרחבה בסעיף 5 של ההצעה.

8. תיאור פרוטוקולי תקשורת

התקשורת במערכת תתבסס על הפרוטוקולים הבאים בשכבות היישום והתעבורה:

1. פרוטוקול HTTP - המערכת תפעל על גבי פרוטוקול HTTPS המאובטח (SSL/TLS).
הבחירה בפרוטוקול זה הכרחית להגנה על פרטיות המשתמש ואבטחת המידע הרגיש (העדפות אישיות, מיקום והיסטוריית הזמנות) העובר בין הלקוח לשרת.
2. פרוטוקול TCP/IP - התעבורה תתבצע על גבי פרוטוקול TCP. נבחר פרוטוקול זה בשל אמינותו והבטחתו כי הנתונים יגיעו בשלמותם ובסדר הנכון. במערכת המבוססת על חישובים אלגוריתמיים מדויקים (כגון הסל החכם), שלמות הנתונים היא קריטית ואינה מאפשרת איבוד חבילות האופייני לפרוטוקול UDP.
מתחת לשכבת ה-TCP, המערכת מתבססת על פרוטוקול האינטרנט (IP) לצורך מיעון וניתוב החבילות ברשת. הפרוטוקול מאפשר לזהות את כתובת השרת ולנתב את הבקשות המגיעות מהדפדפנים השונים אל היעד הנכון, ללא תלות במיקום הפיזי של המשתמש.
3. REST API - במסגרת הפרויקט, המערכת מבצעת תקשורת עם שירותי רשת חיצוניים כדי להעשיר את המידע הזמין לאלגוריתם קבלת ההחלטות. התקשורת מול שירותים אלו מבוססת על פרוטוקול HTTP וארכיטקטורת RESTful API.
המערכת משלבת פיצ'ר מתקדם של המלצות חכמות המותאמות לתנאי הסביבה של המשתמש, המתבסס על שירות המידע המטאורולוגי OpenWeatherMap.
מימוש הפיצ'ר מתבצע באמצעות תקשורת מול ממשק התכנות החיצוני, השרת מתפקד כ"לקוח", יוזם בקשת HTTP GET לשירות המטאורולוגי, ומפענח את אובייקט ה-JSON המתקבל, המכיל את נתוני מזג האוויר במיקום הנוכחי של המשתמש.
מידע זה אינו נשמר במערכת כ"נתון יבש" בלבד, אלא משמש כקלט קריטי למנוע ההמלצות. האלגוריתם מנתח את הטמפרטורה והתחזית ומציף עבור המשתמש באופן אוטומטי פריטי לבוש רלוונטיים (כגון מעילים בימים גשומים או ביגוד קצר בימים חמים). בכך, המערכת הופכת את חווית הקנייה לאישית ומדויקת יותר, וממקסמת את הרלוונטיות של הפריטים המוצעים בסל החכם.

4. פורמט התקשורת (JSON) - כלל המידע העובר בגוף הבקשות ובתוכן התגובות מקודד בפורמט JSON. הבחירה ב-JSON נעשתה בשל היותו פורמט טקסטואלי קליל, קריא לאדם, ונתמך באופן טבעי הן על ידי הדפדפן והן על ידי שרת ה-Spring Boot. פורמט זה

מאפשר העברה יעילה של אובייקטים מורכבים, כגון אובייקט הבקשה המכיל את התקציב, כמות הפריטים והסגנון המועדף.

9. פיתוחים עתידיים

הגרסה הנוכחית של המערכת מהווה "הוכחת היתכנות" ליכולת לבצע אופטימיזציה של סל קניות תחת אילוצים. עם זאת, החזון הטכנולוגי והעסקי של הפרויקט רחב הרבה יותר. להלן כיווני הפיתוח המרכזיים המתוכננים לגרסאות הבאות, אשר לא נכללו בגרסה זו מפאת מסגרת הזמן:

1. מעבר לארכיטקטורת Multi-Vendor

החזון האסטרטגי הוא להרחיב את המערכת מחנות בודדת לפלטפורמת "Super-Aggregator" המאגדת קטלוגים ממספר רב של קמעונאי אונליין (כגון ASOS, ZARA ועוד) באמצעות ממשקי API בזמן אמת. שינוי זה יהפוך את הליבה האלגוריתמית למורכבת ועשירה יותר: המנוע ידרש לשקלל לא רק את מחיר וניקוד הפריט, אלא גם משתנים לוגיסטיים כגון דמי משלוח משתנים וזמני אספקה מספקים שונים. התוצאה תהיה "סל היברידי" אופטימלי, המאפשר למשתמש לרכוש את העסקה המשתלמת ביותר ברשת במקום אחד.

2. פרסונליזציה מבוססת למידת מכונה

בגרסה הנוכחית, ה"ניקוד" לכל פריט הוא סטטי. בפיתוח העתידי ישולב מודל למידת מכונה (Machine Learning) שיחליף את הניקוד הקבוע בניקוד דינמי ומותאם אישית. המערכת תלמד את דפוסי הגלישה, הרכישות והסגנון של כל משתמש, ותייצר פונקציית הערכה (Scoring Function) ייחודית עבורו. כך, שני משתמשים שונים יקבלו המלצות שונות לחלוטין עבור אותו מאגר מוצרים, מה שישיפר דרמטית את אחוזי ההמרה.

3. התמודדות עם עומסים באמצעות אלגוריתמי קירוב

פתרון ה-Knapsack הנוכחי מבוסס על תכנות דינמי ומבטיח פתרון מדויק, אך עשוי להיות כבד חישובית בקלטים גדולים מאוד. כהרחבה טכנולוגית, יפותחו במערכת "אלגוריתמי קירוב" ושיטות חמדניות. מנגנון זה ייכנס לפעולה אוטומטית במקרי קיצון בהם גודל התקציב או כמות הפריטים חורגים מהטווח היעיל, ויספק למשתמש פתרון "כמעט אופטימלי" בזמן אמת, תוך שמירה על ביצועי מערכת גבוהים.

4. אופטימיזציה ירוקה

הרחבת המודל המתמטי כך שישלול מימד נוסף של "ציון אקולוגי" לכל פריט, המבוסס על נתונים כגון הרכב הבד, תהליכי ייצור ומרחק שינוע. המערכת תאפשר למשתמש להוסיף אילוץ ערכי (למשל: "הבטח סל עם דירוג ירוק מעל 80%"). פיתוח זה דורש פתרון בעיית אופטימיזציה רב-ממדית. מציאת האיזון המושלם בין מקסימום סטייל, מינימום מחיר ומינימום פגיעה סביבתית.

10. תיאור טכנולוגיות הנדסה

להלן פירוט סביבת הפיתוח, השפות והכלים הטכנולוגיים שנבחרו למימוש הפרויקט, תוך נימוק הבחירה והתאמתם לדרישות המערכת:

שפת הפיתוח שנבחרה למימוש ליבת המערכת היא Java בגרסה 21, זוהי שפת עילית מבוססת מונחים - OOP, שנבחרה בזכות היציבות, הביצועים הגבוהים והיכולת שלה להתמודד עם לוגיקה מורכבת. גרסה 21 היא גרסה המבטיחה תמיכה ארוכת טווח, וכוללת שיפורי ביצועים בניהול הזיכרון ובכתיבת קוד מודרני ותמציתי יותר.

בסיס המערכת מושתת על מסגרת העבודה Spring Boot בגרסה 3.5.9. מסגרת זו נבחרה כיוון שהיא מספקת תשתית סדורה ומוכחת בתעשייה לבניית יישומי צד שרת מודרניים. השימוש ב-Spring Boot מאפשר ליישם ארכיטקטורה נקייה עם הפרדה ברורה בין השכבות. המסגרת חוסכת זמן פיתוח יקר, ומספקת באופן מובנה מנגנוני אבטחה, ניהול תלויות (Dependency Injection) ושרת Web פנימי, מה שהופך את המערכת לסקיילבילית וקלה לתחזוקה עתידית.

בהתאם לצרכי הפרויקט, שולבו במערכת הספריות הבאות מתוך האקו-סיסטם של Spring:

- **Spring Web:** ספרייה זו מהווה את עמוד השדרה של שכבת התקשורת. היא מספקת את התשתית לפיתוח ה-RESTful API, ומכילה בתוכה את שרת ה-Apache Tomcat המאפשר להריץ את האפליקציה כ-Web Server עצמאי המטפל בבקשות HTTP/HTTPS.
- **Spring Data MongoDB:** ספרייה המשמשת כשכבת ה-ODM. תפקידה הוא לגשר בין קוד ה-Java לבין מסד הנתונים MongoDB. היא מאפשרת לבצע פעולות CRUD ושליפות מורכבות באמצעות ממשקי Repository, ללא צורך בכתיבת שאילתות "נמוכות" ומסובכות, ובכך מיעלת את העבודה מול הנתונים.
- **Vaadin:** פלטפורמה לפיתוח ממשקי WEB עשירים (Full-Stack). הספרייה מאפשרת בניית ממשק משתמש אינטראקטיבי ומודרני בשפת Java, תוך

אינטגרציה חלקה עם ה-Spring Boot Backend, ומספקת רכיבים ויזואליים מוכנים.

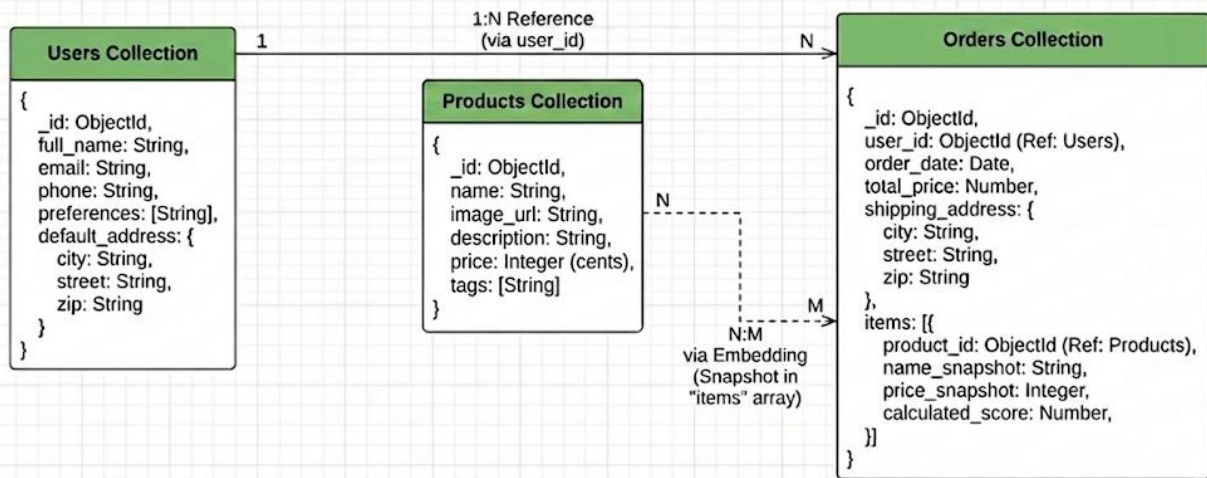
- **Spring Boot DevTools:** סט כלים שנועד לשפר את חווית הפיתוח. הספרייה מאפשרת מנגנוני LiveReload ו-Fast Restart, כך שכל שינוי בקוד מתעדכן מיידית בשרת ללא צורך בהפעלה מחדש מלאה, דבר שמאיץ משמעותית את קצב הפיתוח והבדיקות.
- כלי הפיתוח שנבחר הוא Visual Studio Code. זוהי סביבת פיתוח קלה, מהירה וגמישה, אשר הותאמה לפיתוח ב-Java ו-Spring Boot באמצעות הרחבות ייעודיות. הבחירה ב-VS Code נעשתה בשל הביצועים המהירים שלה והתמיכה הרחבה בכלי דיבאגינג, השלמה אוטומטית וניהול גרסאות, המאפשרים זרימת עבודה חלקה ויעילה.

11. מסד נתונים

- תשתית הנתונים של המערכת מבוססת על מסד נתונים מסוג NoSQL, תוך שימוש בטכנולוגיית MongoDB. בשונה ממסדי נתונים מסורתיים (SQL) הכופים מבנה טבלאי קשיח וקבוע, MongoDB שומר את המידע בתצורה של "אוספים" (Collections) המכילים "מסמכים" (Documents) בפורמט JSON גמיש.
- בחירה הנדסית זו היא קריטית להצלחת הפרויקט מהסיבות הבאות:
- **גמישות הסכמה:** קטלוג מוצרי אופנה מתאפיין בהטרוגניות, זאת אומרת שלפריטים שונים יש תכונות שונות (למשל: לחולצה יש "מידה" ו"סוג בד", בעוד לתיק יש "נפח"). מודל המסמכים מאפשר לשמור כל פריט עם המאפיינים הייחודיים לו באותו אוסף, ללא צורך בנרמול מוגזם או יצירת עמודות ריקות רבות.
 - **טיפול במבנים מורכבים:** המערכת נדרשת לאחסן מערכי תגיות (כגון "חורף", "אלגנט", "צמר") עבור כל פריט לצורך סינון. מסד הנתונים MongoDB תומך באופן טבעי באחסון מערכים ואובייקטים מקוננים, ומאפשר ביצוע שאילתות סינון מהירות עליהם ללא צורך בטבלאות קישור מסובכות.
- מסד הנתונים אינו מתארח מקומית, אלא רץ על גבי תשתית ענן מנוהלת (כגון MongoDB Atlas). ארכיטקטורה זו מבטיחה זמינות גבוהה של המידע מכל מקום, גיבויים אוטומטיים, והגדלת משאבי האחסון והעיבוד בלחיצת כפתור ככל שנפח המשתמשים והמוצרים במערכת יגדל.

להלן תרשים ERD של מסד הנתונים:

NoSQL Document Model Diagram (Smart Cart Project)



הסבר על התרשים:

מודל הנתונים של המערכת מורכב משלושה אוספים (Collections) עיקריים:

- אוסף משתמשים:** ישות זו מאחסנת את פרופיל המשתמש. המסמך כולל מזהה ייחודי, פרטים אישיים (שם, אימייל, טלפון), העדפות אישיות הנשמרות כמערך נתונים אשר ישמשו את המערכת כדי לחשב את הניקוד האישי לכל מוצר בזמן אמת, וכתובת ברירת מחדל למשלוח. מידע רגיש כגון סיסמאות שמורות לאחר הצפנה.
- אוסף מוצרים:** ישות זו מהווה את חומר הגלם המרכזי עבור אלגוריתם ה"סל החכם". כל מסמך באוסף מייצג פריט לבוש, אך בשונה מקטלוגים סטנדרטיים, המיקוד כאן הוא על עושר הנתונים. המסמך מכיל את המידע הלוגיסטי (שם, תמונה, ומחיר המיוצג כמספר שלם), ובנוסף מחזיק מערך מפורט של תגיות ומאפיינים כגון גזרה, סוג בד, צבע מדויק ועונתיות. חשוב לציין כי מבחינה ארכיטקטונית, ישות זו אינה מכילה שדה "ניקוד", שכן הניקוד במערכת זו אינו תכונה סטטית של הבגד, אלא ערך דינמי. הניקוד מחושב על ידי השרת בזמן אמת עבור כל משתמש בנפרד, בהתבסס על מידת ההתאמה בין התגיות של הפריט לבין פרופיל ההעדפות האישי של המשתמש.
- אוסף הזמנות:** ישות זו אחראית על תיעוד הטרנזקציות העסקיות וחיבור בין המשתמש לסל המוצרים שנבחר. מבחינת עיצוב המסד, האוסף ממומש בתבנית של Embedding (הטמעה) המערכת שומרת בתוך ההזמנה "צילום מצב"

(Snapshot) מלא של הפריטים כפי שהיו ברגע הרכישה (כולל המחיר ששולם

בפועל). בנוסף, המבנה תומך בשני תרחישי קנייה:

- **רכישה מבוססת אלגוריתם:** במקרה זה, נשמר לכל פריט גם הניקוד שחושב ברגע יצירת הסל. שמירת הניקוד מאפשרת ניתוח היסטורי ובקרה על החלטות האלגוריתם.
- **רכישה ידנית:** במידה והמשתמש בחר את הפריטים בעצמו ללא עזרת המערכת החכמה, שדה הניקוד יישאר ריק או יסומן כ"ידני", כדי להבדיל בין המלצות המערכת לבין בחירות אישיות עצמאיות.

בתרשים ניתן לראות את הקשרים הלוגיים בין האוספים:

- קיים קשר **N:1** (יחיד לרבים) בין המשתמשים להזמנות, הממומש באמצעות שמירת המפתח (Reference) של המשתמש בתוך מסמך ההזמנה.
- קיים קשר **N:M** (רבים לרבים) בין ההזמנות למוצרים, הממומש באמצעות **Embedding**: בתוך מסמך ההזמנה ישנו מערך items, המכיל אובייקטים המעתיקים את המידע הקריטי מהמוצר (Snapshot) לצורך תיעוד העסקה.

12. פורטים פורמליים

לוח זמנים:

לסיים עד תאריך	שלבי עבודה
1.12.2025	בחירת פרויקט, חקירה ולמידה לעומק של נושאי הפרויקט
11.12.2025	כתיבה והגשת הצעת הפרויקט לאישור משרד החינוך
13.1.2026	מימוש הקוד של האלגוריתם המרכזי, ביצוע בדיקות ושיפורים
3.2.2026	בניית צד שרת
17.2.2026	בניית מסד הנתונים ושילובו
3.3.2026	בניית צד לקוח
24.3.2026	כתיבת ספר הפרויקט
7.4.2026	הגשת הפרויקט כולו (ספר + קוד) להגנה וקבלת ציון מגן

מנחה הפרויקט: מר אילן פרץ

חתימת הסטודנט: 

חתימת רכז המגמה: _____